

Full-Stack Intern Training — Day 6

MODULE

React Core Concepts

AUTHOR

Penumuru Madhu Sudhan Reddy

TRACK

90-Day Full-Stack Web Dev

Core Objective: Master the transition from imperative DOM manipulation to declarative component architecture (UI = f(state)). Understand Virtual DOM diffing, JSX compilation rules, Props vs State boundaries, useState immutability patterns, list key mechanics, and build responsive, type-safe React applications.

1. WHY REACT? IMPERATIVE VS DECLARATIVE PROGRAMMING

To understand React, you must understand the fragility of vanilla JavaScript when applications grow.

The Vanilla JS Problem (Imperative): You must write explicit instructions telling the browser *how* to find, create, mutate, and delete HTML elements line-by-line.

```
// Vanilla JS: Manual DOM manipulation
const input=document.getElementById("task-input");
const addBtn=document.getElementById("add-btn");
const taskList=document.getElementById("task-list");
const countBadge=document.getElementById("count-badge");
let count=0;

addBtn.addEventListener("click",()=>{
  if(!input.value.trim())return;
  const li=document.createElement("li");
  li.textContent=input.value;
  const delBtn=document.createElement("button");
  delBtn.textContent="Delete";
  delBtn.addEventListener("click",()=>{
    taskList.removeChild(li);
    count--;
    countBadge.textContent=count+" tasks";
  });
  li.appendChild(delBtn);
  taskList.appendChild(li);
  count++;
  countBadge.textContent=count+" tasks";
  input.value="";
});
```

VANILLA JS BROWSER OUTPUT (O/P)

User types "Inspect foundation" and clicks [Add]:

- DOM changes: <ul id="task-list">Inspect foundation <button>Delete</button>
- Counter text changes: 1 tasks

Failure mode: If HTML ID changes to tasks-list, JS throws a silent null exception!

The React Way (Declarative): You describe *what* the UI should look like for a given state array, and React handles all DOM creation, updates, and removals automatically.

```
import React,{useState} from 'react';

function TaskApp(){
  const [tasks,setTasks]=useState<string[]>([]);
  const [text,setText]=useState("");

  function addTask(){
    if(!text.trim())return;
    setTasks([...tasks,text]);
    setText("");
  }
  function deleteTask(index:number){
    setTasks(tasks.filter((_,i)=>i!==index));
  }

  return (
    <div>
      <span className="badge">{tasks.length} tasks</span>
      <input value={text} onChange={e=>setText(e.target.value)} placeholder="New task..." />
      <button onClick={addTask}>Add</button>
      <ul>
        {tasks.map((task,i)=>{
          <li key={i}>{task} <button onClick={()=>deleteTask(i)}>Delete</button></li>
        })}
      </ul>
    </div>
  );
}
```

REACT RENDERED OUTPUT (O/P)

Initial: [Badge: 0 tasks] [Input: ""] [Add Button] (empty list)
After adding "Check steel": [Badge: 1 tasks] [Input: ""] [Add Button]
• Check steel [Delete]

2. THE CORE FORMULA: $UI = f(STATE)$ & THE VIRTUAL DOM

React models every view as a mathematical function: $UI = f(state)$.

- `state`: The live data at this instant (e.g. 14 floors completed out of 20).
- `f` (Component): The architect reading the data and returning the blueprint (JSX).
- `UI`: The pixels rendered on screen.

```
// What you write (JSX):  
const element=<div className="tower"><span className="name">Tower A</span></div>;  
  
// Virtual DOM representation in memory (plain JS object):  
const vdomNode={  
  type: 'div',  
  props: {  
    className: 'tower',  
    children: {  
      type: 'span',  
      props: { className: 'name', children: 'Tower A' }  
    }  
  }  
};
```

VIRTUAL DOM INSPECTION O/P

```
console.log(vdomNode.type); // "div"  
console.log(vdomNode.props.className); // "tower"  
console.log(vdomNode.props.children.props.children); // "Tower A"
```

Diffing & Reconciliation: When state updates, React creates a new Virtual DOM tree, diffs it against the old tree in milliseconds, finds the exact text node that changed, and patches only that single real DOM node!

3. JSX DEEP DIVE: COMPILATION & THE 4 GOLDEN RULES

JSX is syntactic sugar that compiles to `React.createElement()` function calls.

```
// Input JSX:
<header className="header"><h1>BuildTrack</h1></header>

// Transpiled JavaScript output:
React.createElement("header", { className: "header" }, React.createElement("h1", null, "BuildTrack"));
```

The 4 Golden Rules:

- Single Root Element / Fragment:** Return `<>...</>` to avoid adding unnecessary wrapper `<div>` nodes to the DOM.
- camelCase Attributes:** `class` becomes `className`, `for` becomes `htmlFor`, and inline styles use objects: `style={{ backgroundColor: '#fff', fontSize: '14px' }}`.
- Self-Closing Tags:** All empty tags MUST close: `<input />`, `<img />`, `<br />`.
- Curly Braces { }**: Switches from JSX back to normal JavaScript expressions (variables, math, ternaries, and function calls).

```
function FloorBadge({ floor, total }: { floor: number; total: number }) {
  const isTopFloor = floor === total;
  return (
    <div>
      <h3>Floor {floor} of {total}</h3>
      <p>Progress: {(floor/total)*100}%</p>
      <span>Status: {isTopFloor ? "Penthouse Level" : "Standard Floor"}</span>
    </div>
  );
}
```

RENDERED OUTPUT FOR `<FLOORBADGE FLOOR={10} TOTAL={10} />` (O/P)

```
<div>
  <h3>Floor 10 of 10</h3>
  <p>Progress: 100%</p>
  <span>Status: Penthouse Level</span>
</div>
```

4. PROPS VS STATE: CORE COMPARISON & DATA FLOW

Feature	Props	State
Definition	Input data passed from parent component	Internal data managed within component
Ownership	Owned and passed by parent	Owned by the component calling <code>useState</code>
Mutability	Read-only (Immutable)	Mutable via setter function (<code>setState</code>)
Trigger Render?	Yes, when parent passes updated props	Yes, when setter function is called
TypeScript	<code>interface CardProps { title: string; }</code>	<code>const [count, setCount] = useState(0)</code>

Data Flow: Flows strictly DOWN from parent to child. To communicate upwards, parent passes a callback function via props.

```
interface WorkerCardProps {
  name: string;
  role: string;
  isAvailable: boolean;
  onToggleStatus: (name: string) => void;
}

function WorkerCard({ name, role, isAvailable, onToggleStatus }: WorkerCardProps) {
  return (
    <div style={{ border: '1px solid #ccc', padding: '8px', margin: '4px 0' }}>
      <h4 style={{ margin: '0 0 4px 0' }}>{name} ({role})</h4>
      <span>Status: {isAvailable ? "On Site" : "Off Duty"}</span>
      <button onClick={() => onToggleStatus(name)} style={{ marginLeft: '10px' }}>Toggle</button>
    </div>
  );
}
```

WORKERCARD RENDERED O/P

```
Ramesh Kumar (Site Supervisor)
Status: On Site [Toggle]
Clicking [Toggle] calls props.onToggleStatus("Ramesh Kumar"), alerting parent to update state.
```

5. DEEP DIVE INTO USESTATE: MEMORY SNAPSHOTS & IMMUTABILITY

Syntax: `const [state, setState] = useState(initialValue);`

A. State is a Photographic Snapshot

```
function Counter(){
  const [count, setCount]=useState(0);
  function handleClick(){
    console.log("Before:", count);
    setCount(count+1);
    console.log("After:", count); // What prints?
  }
  return <button onClick={handleClick}>Count: {count}</button>;
}
```

CONSOLE & UI OUTPUT ON CLICK (O/P)

Console: Before: 0
 Console: After: 0 (State update is scheduled for next render, does not mutate local variable!)
 Rendered Button: Count: 1

B. The Law of Immutability

Never mutate state directly! React performs reference equality checks (`oldState === newState`). If the pointer doesn't change, React skips rendering.

Action	✗ Mutating (Avoid!)	✓ Immutable (Correct!)
Add Item	<code>arr.push(item)</code>	<code>[...arr, item]</code>
Remove Item	<code>arr.splice(index, 1)</code>	<code>arr.filter((_, i) => i !== index)</code>
Update Item	<code>arr[i].done = true</code>	<code>arr.map((item, i) => i === index ? { ...item, done: true } : item)</code>
Update Object	<code>obj.role = "Admin"</code>	<code>{ ...obj, role: "Admin" }</code>

C. Functional State Updates (prev => ...)

When next state depends on current state, use the updater function form:

```
function BatchDemo(){
  const [count, setCount]=useState(0);
  function handleTriple(){
    setCount(prev=>prev+1); // receives 0 -> returns 1
    setCount(prev=>prev+1); // receives 1 -> returns 2
    setCount(prev=>prev+1); // receives 2 -> returns 3
  }
  return <button onClick={handleTriple}>Tripled: {count}</button>;
}
```

OUTPUT ON CLICK (O/P)

Button text changes from 0 to 3 in a single clean render pass!

6. THE RE-RENDER CYCLE: STEP-BY-STEP

1. **User Event:** User clicks a button or types.
2. **State Setter:** Component event handler calls `setState(newValue)`.
3. **Render Phase:** React executes the component function from line 1 to end, recalculating variables and returning a fresh Virtual DOM.
4. **Reconciliation (Diffing):** React compares new VDOM vs old VDOM to isolate changes.
5. **Commit Phase:** React patches only the updated elements in the real DOM.
6. **Repaint:** Browser paints the updated pixels smoothly.

7. CONDITIONAL RENDERING & THE "0" GOTCHA

There are 3 standard conditional rendering patterns in React:

```
// 1. Ternary Operator (Choose between A or B)
{isApproved ? <span style={{ color: 'green' }}>Approved</span> : <span style={{ color: 'orange' }}>Pending</span>}

// 2. Logical AND && (Show or Hide)
{hasHazard && <div className="hazard-warning">⚠️ High winds on crane!</div>}

// 3. Early Return (Guard Clause)
if(isLoading){
  return <div>Loading site metrics...</div>;
}
```

⚠️ The Dangerous Number "0" Bug:

In JavaScript, `0 && <div />` evaluates to `0` (falsy). React renders the literal string `"0"` into the DOM!

```
// ❌ BUG: {tasks.length && <p>Tasks exist</p>} -> Renders "0" when array is empty!
// ✅ FIX: {tasks.length > 0 && <p>Tasks exist</p>} or {Boolean(tasks.length) && <p>...</p>}
```

8. LIST RENDERING & WHY "KEY" MATTERS

Render arrays using `.map()`. Always provide a stable, unique `key` (e.g. `task.id`):

```
const towers=[
  { id: "tow-a", name: "Tower Alpha", floors: 24 },
  { id: "tow-b", name: "Tower Beta", floors: 18 }
];

return (
  <ul>
    {towers.map(t=>(
      <li key={t.id}><strong>{t.name}</strong> - {t.floors} Floors</li>
    ))}
  </ul>
);
```

RENDERED OUTPUT (O/P)

- Tower Alpha - 24 Floors
- Tower Beta - 18 Floors

The Index-as-Key Trap: If you use `key={index}` and delete an item, React matches keys by position. Form inputs and checked checkboxes from deleted items will mistakenly attach to neighboring elements!

9. CONTROLLED COMPONENTS & FORM ARCHITECTURE

An input whose value is bound to and updated by React state is a "Controlled Component".

```
function MaterialForm(){
  const [material,setMaterial]=useState("");
  function handleSubmit(e: React.FormEvent){
    e.preventDefault(); // Prevents full-page browser refresh!
    console.log("Saved:", material);
    setMaterial("");
  }
  return (
    <form onSubmit={handleSubmit}>
      <input value={material} onChange={e=>setMaterial(e.target.value)} placeholder="Material..." />
      <button type="submit">Save</button>
    </form>
  );
}
```

INTERACTION & CONSOLE OUTPUT (O/P)

- User types "Cement 50kg" and hits [Enter]:
- Browser does NOT reload.
 - Console prints: "Saved: Cement 50kg"
 - Input field automatically resets to empty string "".

10. LINE-BY-LINE ARCHITECTURE OF DAY 6 EXERCISES

Component	File	Core Logic & Operations	Expected Output (o/p)
<TaskList />	exercise-1-task-list.tsx	• <code>addTask</code> : <code>setTasks([...tasks, { id: Date.now(), text, done: false }])</code> • <code>toggleTask</code> : <code>tasks.map(t => t.id===id ? { ...t, done: !t.done } : t)</code> • <code>deleteTask</code> : <code>tasks.filter(t => t.id!==id)</code> • <code>filter</code> : Active / Completed tabs	Interactive task board with count badges, strikethrough completed styling, and dynamic filters.
<ProgressBar />	exercise-2-progress-bar.tsx	• Clamps percentage: <code>Math.min(100, Math.max(0, percentage))</code> • Risk color lookup: <code>'on-track': '#10b981', 'at-risk': '#f59e0b', 'high-risk': '#ef4444', 'critical': '#dc2626'</code> • Smooth CSS width transition.	A progress bar filling from 0% to 100% with real-time risk level color shift and percentage label.
<FloorGrid />	exercise-3-floor-grid.tsx	• 20-floor state array. • Status cyler: <code>not-started → in-progress → completed → not-started</code> . • Summary stats: <code>floors.filter(...).length</code> . • 5x4 CSS Grid layout.	5x4 interactive grid of floor buttons. Clicking any floor cycles color: gray → amber → green and updates site percentage.

SELF-ASSESSMENT CHECKLIST

1. **Why does mutating state directly fail to re-render?** Because array/object memory references remain identical, causing React's shallow comparison (`old === new`) to evaluate to `true` and skip rendering.
2. **What does `UI = f(state)` mean?** The user interface is a pure visual reflection of the underlying state data, recomputed whenever state changes.
3. **Why use unique IDs instead of index for keys?** Array indexes shift when items are deleted or prepended, which causes React to reuse mismatched DOM state like form inputs and checkboxes.
4. **How do child components send updates to parent components?** By invoking callback functions passed down from the parent via props.